

Estruturas de Dados Elementares*

PCC104 – Projeto e Análise de Algoritmos

18 de maio de 2026

Sumário

1	Vetores	1
2	Matrizes (Tensores)	2
3	Pilhas	2
4	Filas	3
5	Listas Encadeadas	4
5.1	Definição	4
5.2	Busca	4
5.3	Inserção	4
5.4	Remoção	5
5.5	Sentinelas	5
6	Árvores Enraizadas	5
6.1	Árvores Binárias	5
6.2	Árvores Enraizadas com Ramificações Ilimitadas	7

1 Vetores

Um **vetor** (ou *array*) é a estrutura de dados mais primitiva e mais importante da computação: um bloco contíguo de memória dividido em posições de mesmo tamanho, cada uma identificada por um índice inteiro. A organização contígua é o que dá ao vetor a sua propriedade definidora — o **acesso aleatório em tempo constante**.

Definição 1.1 (Vetor). Um vetor A de tamanho n é uma sequência $A[1], A[2], \dots, A[n]$ de elementos do mesmo tipo, armazenados em posições consecutivas de memória. Dado um endereço-base $\text{base}(A)$ e o tamanho s de cada elemento, o endereço do i -ésimo elemento é

$$\text{end}(A[i]) = \text{base}(A) + (i - 1) \cdot s.$$

Operações fundamentais. As três operações elementares — ACESSO, ATRIBUIÇÃO e COMPRIMENTO — executam em tempo $\Theta(1)$. A inserção e remoção *no meio* do vetor, em contrapartida, exigem deslocar todos os elementos posteriores, custando $\Theta(n)$ no pior caso.

Algorithm 1 $\text{INSERIR}(A, n, i, x)$ – insere x na posição i de um vetor de tamanho n

Entrada: Vetor A com capacidade $\geq n + 1$, índice $1 \leq i \leq n + 1$, valor x

```
1: para  $j = n$  decrescendo até  $i$  faça
2:    $A[j + 1] \leftarrow A[j]$ 
3: fim para
4:  $A[i] \leftarrow x$ 
5: retorna  $n + 1$ 
```

Indexação base-0 vs. base-1. O CLRS adota indexação base-1 por clareza pedagógica, mas **linguagens reais** (C, Java, Python) **usam base-0**. A fórmula de endereçamento em base-0 é simplesmente $\text{end}(A[i]) = \text{base}(A) + i \cdot s$.

Exemplo 1.1 (Custo do acesso aleatório). Calcular o endereço de $A[i]$ envolve uma soma e uma multiplicação — operações de custo fixo, independentes de n . É por isso que dizemos que $A[i]$ tem custo $O(1)$, e essa propriedade é a fundação de muitos algoritmos eficientes.

Observação 1.1. Embora o acesso seja constante, **percorrer** um vetor é $\Theta(n)$. A diferença é sutil mas crucial: o *acesso* a uma posição é barato; a *varredura* do vetor inteiro não é.

*Material baseado em (?).

2 Matrizes (Tensores)

Uma **matriz** é uma generalização bidimensional do vetor, e um **tensor** generaliza a noção para k dimensões. Conceitualmente, uma matriz A de dimensões $m \times n$ é um arranjo retangular de elementos $A[i][j]$ com $1 \leq i \leq m$ e $1 \leq j \leq n$.

Esquemas de armazenamento. A memória do computador é unidimensional, então uma matriz precisa ser *linearizada*. Duas convenções dominam:

- **Row-major** (C, C++, Python/NumPy padrão): os elementos de uma mesma linha ficam contíguos.

$$\text{end}(A[i][j]) = \text{base}(A) + ((i-1) \cdot n + (j-1)) \cdot s$$

- **Column-major** (Fortran, MATLAB, Julia): os elementos de uma mesma coluna ficam contíguos.

$$\text{end}(A[i][j]) = \text{base}(A) + ((j-1) \cdot m + (i-1)) \cdot s$$

Observação 2.1 (Localidade de cache). A escolha do esquema afeta **drasticamente** o desempenho prático. Percorrer uma matriz em ordem oposta à do armazenamento gera *cache misses* a cada acesso. Em row-major, sempre prefira o laço externo sobre as linhas:

```
for i: for j: ...A[i][j]... (rápido em row-major)
for j: for i: ...A[i][j]... (lento em row-major)
```

A complexidade assintótica é a mesma, mas a constante escondida pode variar em uma ordem de magnitude.

Tensores. Um tensor de ordem k com dimensões $d_1 \times d_2 \times \dots \times d_k$ generaliza o cálculo de endereço. Em row-major,

$$\text{end}(A[i_1, i_2, \dots, i_k]) = \text{base}(A) + \left(\sum_{r=1}^k (i_r - 1) \prod_{q=r+1}^k d_q \right) \cdot s.$$

Exemplo 2.1 (Multiplicação de matrizes – versão ingênua). O algoritmo direto tem três laços aninhados e custo $\Theta(n^3)$:

Algorithm 2 MULT-MAT(A, B, n)

Entrada: Matrizes A, B de dimensão $n \times n$

```
1: Seja  $C$  uma matriz  $n \times n$  inicializada com zeros
2: para  $i = 1$  até  $n$  faça
3:   para  $j = 1$  até  $n$  faça
4:     para  $k = 1$  até  $n$  faça
5:        $C[i][j] \leftarrow C[i][j] + A[i][k] \cdot B[k][j]$ 
6:     fim para
7:   fim para
8: fim para
9: retorna  $C$ 
```

A ordem dos laços não muda a corretude nem o custo assintótico, mas afeta drasticamente o desempenho de cache. Algoritmos sub-cúbicos (Strassen $\Theta(n^{\log_2 7})$) serão discutidos adiante.

3 Pilhas

Uma **pilha** (*stack*) é uma estrutura de dados que implementa a disciplina **LIFO** (*Last-In, First-Out*): o último elemento inserido é o primeiro a ser removido. A analogia clássica é uma pilha de pratos — só se pode adicionar ou remover do topo.

Definição 3.1 (Pilha). Uma pilha S suporta três operações:

- EMPILHA(S, x) – insere x no topo de S ;
- DESEMPILHA(S) – remove e retorna o elemento do topo;
- PILHA-VAZIA(S) – retorna verdadeiro se S não contém elementos.

Implementação com vetor. Mantemos um vetor $S[1 \dots n]$ e um atributo $S.\text{topo}$ indicando a posição do elemento mais recente. Convenciona-se $S.\text{topo} = 0$ para a pilha vazia.

Algorithm 3 PILHA-VAZIA(S)

```
1: se  $S.\text{topo} = 0$  então
2:   retorna verdadeiro
3: senão
4:   retorna falso
5: fim se
```

Algorithm 4 EMPILHA(S, x)

```
1:  $S.topo \leftarrow S.topo + 1$   
2:  $S[S.topo] \leftarrow x$ 
```

Algorithm 5 DESEMPILHA(S)

```
1: se PILHA-VAZIA( $S$ ) então  
2:   erro “estouro inferior”  
3: senão  
4:    $S.topo \leftarrow S.topo - 1$   
5:   retorna  $S[S.topo + 1]$   
6: fim se
```

Análise. As três operações executam em tempo $O(1)$. O custo é literalmente uma incrementação ou decrementação seguida de uma atribuição ou leitura.

Observação 3.1 (Estouro de pilha). Se $S.topo$ exceder n , ocorre *overflow* (estouro superior). Na prática, esse erro pode ser tratado com *realocação dinâmica* do vetor, dobrando sua capacidade — uma técnica que estudaremos como *tabela dinâmica*, cujo custo amortizado por operação permanece $O(1)$.

Exemplo 3.1 (Aplicação: avaliação de expressões). Pilhas são a estrutura natural para avaliar expressões em *notação pós-fixada* (notação polonesa reversa). Por exemplo, $3\ 4 + 2 *$ corresponde a $(3 + 4) \cdot 2 = 14$. Compiladores também usam pilhas para gerenciar chamadas de função (a chamada *pilha de execução*).

4 Filas

Uma *fila* (*queue*) implementa a disciplina **FIFO** (*First-In, First-Out*): o primeiro elemento inserido é o primeiro a ser removido — como uma fila de banco.

Definição 4.1 (Fila). Uma fila Q suporta duas operações principais:

- ENFILEIRA(Q, x) — insere x no final (*cauda*) de Q ;
- DESENFILEIRA(Q) — remove e retorna o elemento do início (*cabeça*) de Q .

Implementação com vetor circular. Uma fila com $n - 1$ elementos no máximo pode ser implementada com um vetor $Q[1 \dots n]$ e dois atributos: $Q.cabeça$ aponta para o primeiro elemento, e $Q.cauda$ aponta para a próxima posição livre. Para evitar deslocamentos, o vetor é tratado como **circular**: após a posição n , voltamos à posição 1.

A fila está vazia quando $Q.cabeça = Q.cauda$ e cheia quando $Q.cabeça = Q.cauda + 1 \pmod{n}$. Sacrificamos uma posição do vetor para distinguir os dois casos.

Algorithm 6 ENFILEIRA(Q, x)

```
1:  $Q[Q.cauda] \leftarrow x$   
2: se  $Q.cauda = Q.comprimento$  então  
3:    $Q.cauda \leftarrow 1$   
4: senão  
5:    $Q.cauda \leftarrow Q.cauda + 1$   
6: fim se
```

Algorithm 7 DESENFILEIRA(Q)

```
1:  $x \leftarrow Q[Q.cabeça]$   
2: se  $Q.cabeça = Q.comprimento$  então  
3:    $Q.cabeça \leftarrow 1$   
4: senão  
5:    $Q.cabeça \leftarrow Q.cabeça + 1$   
6: fim se  
7: retorna  $x$ 
```

Análise. Ambas as operações executam em tempo $O(1)$.

Observação 4.1 (Por que circular?). Sem o tratamento circular, $Q.cauda$ avançaria indefinidamente até o fim do vetor, mesmo com espaço livre no início após sucessivas remoções. O **módulo** permite reaproveitar o espaço liberado.

Exemplo 4.1 (Variações). Filas dão origem a estruturas mais sofisticadas:

- *Deque (double-ended queue)*: permite inserção e remoção em ambas as extremidades.
- *Fila de prioridades*: cada elemento tem uma prioridade; o de maior (ou menor) prioridade é removido primeiro. Implementada eficientemente com *heaps* — assunto de aula futura.

5 Listas Encadeadas

A diferença fundamental entre listas encadeadas e vetores é a **forma de organização na memória**: enquanto vetores ocupam posições contíguas, listas encadeadas usam **ponteiros** para conectar elementos espalhados pela memória. Esse desacoplamento é o que permite inserções e remoções eficientes — ao custo de perder o acesso aleatório.

5.1 Definição

Definição 5.1 (Lista duplamente encadeada). Uma lista duplamente encadeada L é uma coleção de objetos (*nós*), cada um contendo:

- um campo *chave* (*key*) — o dado armazenado;
- um ponteiro *prev* — para o nó anterior, ou NIL se for o primeiro;
- um ponteiro *next* — para o nó seguinte, ou NIL se for o último.

A lista é referenciada por um atributo $L.cabeca$, que aponta para o primeiro nó (ou NIL se a lista é vazia).

Variantes. Quatro variantes são comuns na prática:

- **Simplesmente encadeada**: cada nó tem apenas *next*. Economiza memória, mas remover um nó dado requer um ponteiro auxiliar para o anterior.
- **Duplamente encadeada**: cada nó tem *prev* e *next*.
- **Ordenada**: as chaves aparecem em ordem.
- **Circular**: o *next* do último nó aponta para o primeiro (e o *prev* do primeiro aponta para o último, no caso duplamente encadeado).

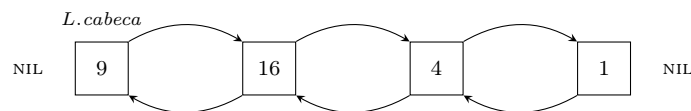


Figura 1: Lista duplamente encadeada com chaves $\langle 9, 16, 4, 1 \rangle$.

5.2 Busca

A busca em uma lista encadeada é *linear*: percorremos os nós sequencialmente até encontrar a chave desejada ou chegar ao fim.

Algorithm 8 LISTA-BUSCA(L, k)

```

1:  $x \leftarrow L.cabeca$ 
2: enquanto  $x \neq \text{NIL}$  e  $x.chave \neq k$  faça
3:    $x \leftarrow x.next$ 
4: fim enquanto
5: retorna  $x$ 
```

Análise. No pior caso, a busca examina todos os n nós antes de falhar ou de encontrar a chave na última posição. O tempo é $\Theta(n)$. Note que, ao contrário do vetor, **não há busca binária em listas encadeadas** — a ausência de acesso aleatório torna inviável saltar para a posição central.

5.3 Inserção

Inserir um nó no início de uma lista duplamente encadeada exige apenas a manipulação de quatro ponteiros, independentemente do tamanho da lista.

Algorithm 9 LISTA-INSERE(L, x) — insere x no início de L

```

1:  $x.next \leftarrow L.cabeca$ 
2: se  $L.cabeca \neq \text{NIL}$  então
3:    $L.cabeca.prev \leftarrow x$ 
4: fim se
5:  $L.cabeca \leftarrow x$ 
6:  $x.prev \leftarrow \text{NIL}$ 
```

Análise. Tempo $O(1)$. Note que isso vale para inserção em uma [posição conhecida](#); inserir mantendo ordenação ou em uma posição arbitrária dada por chave requer busca prévia, totalizando $\Theta(n)$.

5.4 Remoção

Para remover um nó x , basta religar os ponteiros dos vizinhos, tomando cuidado com os casos de borda (quando x é o primeiro ou o último nó).

Algorithm 10 LISTA-REMOVE(L, x) – remove x de L

```
1: se  $x.prev \neq \text{NIL}$  então
2:    $x.prev.next \leftarrow x.next$ 
3: senão
4:    $L.cabeca \leftarrow x.next$ 
5: fim se
6: se  $x.next \neq \text{NIL}$  então
7:    $x.next.prev \leftarrow x.prev$ 
8: fim se
```

Análise. A remoção propriamente dita é $O(1)$, dado o ponteiro x . Mas se a entrada for uma *chave*, é necessário primeiro buscar o nó, totalizando $\Theta(n)$ no pior caso.

5.5 Sentinelas

O código de LISTA-REMOVE tem oito linhas, das quais cinco são testes para tratar os **casos de borda** (nó na cabeça, nó na cauda). Sentinelas eliminam essas verificações com um truque elegante.

Definição 5.2 (Sentinela). Uma *sentinela* é um nó *dummy* que não armazena dado relevante, mas serve para simplificar a estrutura. Em uma lista duplamente encadeada *circular com sentinela*, um único nó $L.nil$ fica entre o último e o primeiro elementos:

- $L.nil.next$ aponta para o primeiro nó real (cabeça);
- $L.nil.prev$ aponta para o último nó real (cauda);
- em uma lista vazia, $L.nil.next = L.nil.prev = L.nil$.

Com essa convenção, a remoção fica radicalmente mais simples:

Algorithm 11 LISTA-REMOVE'(L, x) – versão com sentinela

```
1:  $x.prev.next \leftarrow x.next$ 
2:  $x.next.prev \leftarrow x.prev$ 
```

Sem testes, sem casos de borda. A inserção também simplifica analogamente.

Observação 5.1 (Quando usar sentinelas). Sentinelas reduzem constantes ocultas e tornam o código mais limpo, mas [ocupam memória adicional](#). Para muitas listas pequenas, o custo agregado é apreciável. A regra prática: use sentinelas quando a estrutura é uma só e razoavelmente grande, ou quando a clareza do código for crítica.

6 Árvores Enraizadas

Vetores, listas, pilhas e filas são estruturas *lineares*. **Árvores** são a primeira generalização hierárquica que estudaremos – uma família de estruturas onde cada elemento pode ter múltiplos sucessores. Sua importância em algoritmos é difícil de exagerar: estão por trás de árvores de busca binária, heaps, árvores B, tries, árvores de decisão, e da própria estrutura de chamadas recursivas.

Da mesma forma que listas encadeadas, árvores são tipicamente implementadas com **nós conectados por ponteiros**. A questão principal é [quantos ponteiros cada nó precisa](#), e a resposta depende do número de filhos.

6.1 Árvores Binárias

Definição 6.1 (Árvore binária). Uma árvore binária T é uma coleção de nós em que cada nó x contém:

- $x.chave$ – o dado armazenado;
- $x.p$ – ponteiro para o nó *pai* (NIL se x for a raiz);
- $x.esq$ – ponteiro para o filho à esquerda (NIL se inexistente);
- $x.dir$ – ponteiro para o filho à direita (NIL se inexistente).

A árvore é acessada pelo atributo $T.raiz$, que aponta para o nó raiz (ou NIL se a árvore é vazia).

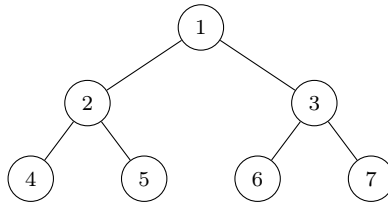


Figura 2: Uma árvore binária com 7 nós e altura 2.

Terminologia.

- A **raiz** é o único nó sem pai.
- Um nó sem filhos é uma **folha**; os demais são **nós internos**.
- O **nível** (ou *profundidade*) de um nó é o número de arestas no caminho até a raiz. A raiz está no nível 0.
- A **altura** de um nó é o número de arestas no caminho mais longo até uma folha sob ele. A altura da árvore é a altura da sua raiz.
- Uma árvore binária está **cheia** quando todo nó interno tem exatamente dois filhos.
- Está **completa** quando todos os níveis estão preenchidos, exceto possivelmente o último, que é preenchido da esquerda para a direita.

Teorema 6.1 (Limite de nós). *Uma árvore binária de altura h tem no máximo $2^{h+1} - 1$ nós.*

Demonstração. Por indução em h . Para $h = 0$, a árvore tem apenas a raiz, isto é, $2^{0+1} - 1 = 1$ nó. Suponha o resultado válido para altura $h - 1$: uma árvore com altura $h - 1$ tem no máximo $2^h - 1$ nós. Uma árvore de altura h é a raiz mais duas subárvores de altura no máximo $h - 1$, logo tem no máximo

$$1 + 2(2^h - 1) = 2^{h+1} - 1 \text{ nós.}$$

□

Corolário 6.2. *Uma árvore binária com n nós tem altura $h \geq \lceil \log_2(n + 1) \rceil - 1$. Em particular, $h = \Omega(\log n)$.*

Esse corolário é a justificativa fundamental para árvores **balanceadas**: se conseguirmos manter $h = O(\log n)$, operações que percorrem um caminho da raiz a uma folha (busca, inserção, remoção em BSTs) executam em tempo $O(\log n)$.

Percursos. Os três percursos canônicos de uma árvore binária são definidos recursivamente:

Algorithm 12 EM-ORDEM(x) – visita: esquerda, raiz, direita

```

1: se  $x \neq \text{NIL}$  então
2:   EM-ORDEM( $x.esq$ )
3:   imprime  $x.chave$ 
4:   EM-ORDEM( $x.dir$ )
5: fim se

```

Algorithm 13 PRÉ-ORDEM(x) – visita: raiz, esquerda, direita

```

1: se  $x \neq \text{NIL}$  então
2:   imprime  $x.chave$ 
3:   PRÉ-ORDEM( $x.esq$ )
4:   PRÉ-ORDEM( $x.dir$ )
5: fim se

```

Algorithm 14 PÓS-ORDEM(x) – visita: esquerda, direita, raiz

```

1: se  $x \neq \text{NIL}$  então
2:   PÓS-ORDEM( $x.esq$ )
3:   PÓS-ORDEM( $x.dir$ )
4:   imprime  $x.chave$ 
5: fim se

```

Análise dos percursos. Cada nó é visitado exatamente uma vez e cada visita realiza trabalho $O(1)$ (excluindo as chamadas recursivas). Portanto, qualquer dos três percursos executa em **tempo $\Theta(n)$** para uma árvore com n nós.

Exemplo 6.1 (Aplicação dos percursos).

- *Em-ordem* em uma árvore de busca binária imprime as chaves em ordem crescente.

- *Pós-ordem* é usada para liberar a memória de uma árvore (filhos antes do pai) e para avaliação de expressões aritméticas representadas como árvores.
- *Pré-ordem* é natural para serializar a estrutura da árvore.

6.2 Árvores Enraizadas com Ramificações Ilimitadas

Para árvores binárias, dois ponteiros por nó bastam. Mas e quando o número de filhos é arbitrário e variável – como em árvores de sistema de arquivos, árvores de derivação sintática, ou árvores B com até centenas de filhos por nó? A solução ingênua de manter um vetor de ponteiros de tamanho fixo desperdiça memória; uma alocação dinâmica complica a estrutura.

Representação por filho-esquerdo / irmão-direito. Existe uma **representação elegante** que usa apenas dois ponteiros por nó, independentemente do número de filhos:

Definição 6.2 (Representação filho-esquerdo / irmão-direito). Cada nó x contém:

- $x.p$ – ponteiro para o pai;
- $x.filho-esq$ – ponteiro para o **filho mais à esquerda** de x ;
- $x.irmao-dir$ – ponteiro para o **irmão imediatamente à direita** de x .

Se x não tem filhos, $x.filho-esq = \text{NIL}$; se x é o filho mais à direita do seu pai, $x.irmao-dir = \text{NIL}$.

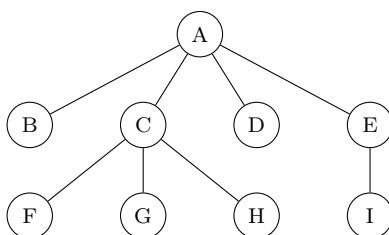


Figura 3: Árvore com ramificação ilimitada. Cada nó pode ter qualquer número de filhos.

A figura acima representa uma árvore onde a raiz A tem 4 filhos e C tem 3 filhos. Na representação filho-esquerdo / irmão-direito, os filhos de cada nó formam uma **lista encadeada simples horizontalmente**, ligada pelo ponteiro *irmão-direito*, e o nó pai mantém apenas o ponteiro para o início dessa lista.

Acesso aos filhos. Para iterar pelos filhos de um nó x :

Algorithm 15 PERCORRE-FILHOS(x)

```

1:  $y \leftarrow x.filho-esq$ 
2: enquanto  $y \neq \text{NIL}$  faça
3:   processa  $y$ 
4:    $y \leftarrow y.irmao-dir$ 
5: fim enquanto

```

Observação 6.1 (Espaço). A grande vantagem dessa representação é que o espaço por nó é **constante** – três ponteiros, independentemente do grau. O preço é que o acesso ao k -ésimo filho é $\Theta(k)$ e não $\Theta(1)$, mas em muitos algoritmos isso é irrelevante porque os filhos são processados sequencialmente.

Outras representações. Dependendo da aplicação, outras estratégias são úteis:

- **Vetor de filhos:** cada nó contém um vetor (ou lista) dinâmico de ponteiros. Acesso ao k -ésimo filho em $\Theta(1)$, mas custo de memória adicional.
- **Representação por pais:** cada nó armazena apenas $x.p$. Suficiente para representar a estrutura, mas inviabiliza percursos descendentes. Útil em estruturas *union-find*.
- **Representação implícita (array-based):** para árvores binárias completas, podemos armazenar os nós em um vetor $A[1 \dots n]$ com a convenção de que o pai de $A[i]$ é $A[\lfloor i/2 \rfloor]$ e os filhos são $A[2i]$ e $A[2i + 1]$. **Nenhum ponteiro é necessário** – essa é a representação usada em *heaps*.

Exemplo 6.2 (Heaps). Uma fila de prioridades implementada com *heap binário* usa um vetor para representar uma árvore binária quase-completa. Operações de inserção e extração executam em $O(\log n)$ percorrendo um único caminho da raiz a uma folha. Veremos heaps em detalhe na próxima aula.

Resumo comparativo. As estruturas estudadas neste capítulo cobrem dois eixos: organização (linear vs. hierárquica) e implementação (contígua vs. encadeada). A tabela abaixo resume os custos das operações típicas.

Estrutura	Acesso	Busca	Inserção	Remoção
Vetor	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Pilha (topo)	$O(1)$	–	$O(1)$	$O(1)$
Fila (extremidades)	$O(1)$	–	$O(1)$	$O(1)$
Lista encadeada	$O(n)$	$O(n)$	$O(1)^\dagger$	$O(1)^\dagger$
Árvore binária	$O(n)$	$O(n)$	$O(n)$	$O(n)$

[†] Dado o ponteiro para a posição de inserção/remoção.

A tabela deixa clara a motivação dos próximos capítulos: para obter **busca, inserção e remoção todas em $O(\log n)$** , precisaremos de árvores *balanceadas* – BSTs balanceadas, árvores rubro-negras, árvores B – estudadas nas próximas aulas.